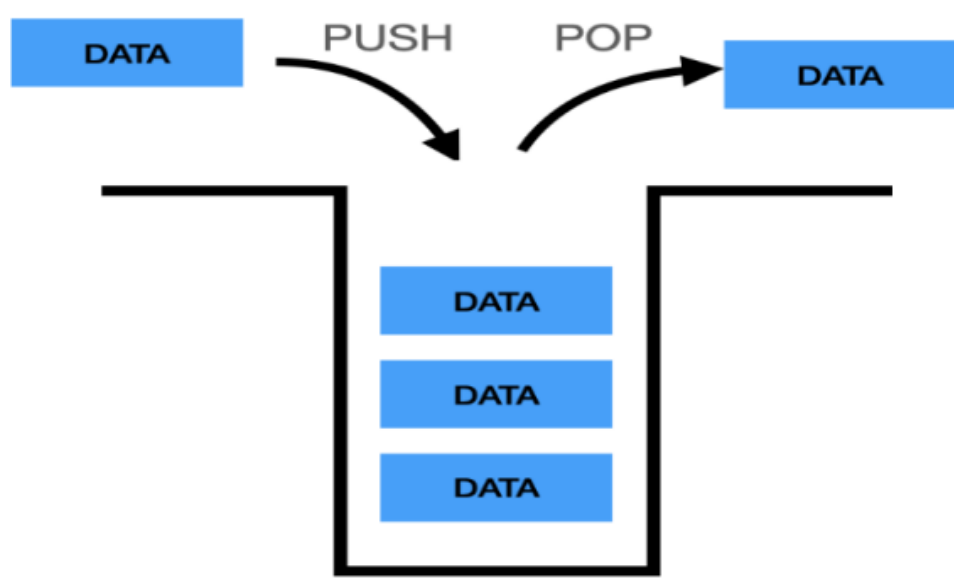


파이썬 프로그래밍 4주차 요약본

16. 알고리즘

16.1 스택(Stack)

- 데이터를 처리하는 기본 자료구조
- 데이터의 삽입과 삭제가 상단(top)에서만 발생
- 후입선출(Last In, First Out, LIFO)
 - 가장 마지막에 입력된 데이터가 가장 먼저 제거되는 구조
 - push : 스택에 데이터를 삽입하는 동작
 - pop : 스택에서 데이터를 제거하는 동작



16.2 큐(Queue)

- 데이터를 처리하는 기본 자료구조
- 선입선출(First In, First Out, FIFO)
 - 마지막에 데이터를 입력하고 가장 먼저 입력된 데이터가 먼저 제거되는 구조

- Enqueue : 큐에 데이터를 삽입하는 동작
- Dequeue : 큐에서 데이터를 제거하는 동작



16.3 덱(Deque)

- 스택과 큐를 병합한 형태의 자료구조
- 양쪽 끝에서 삽입과 삭제가 가능한 자료구조
 - append : 덱의 마지막에 데이터를 삽입하는 동작
 - appendleft : 덱의 처음에 데이터를 삽입하는 동작
 - pop : 덱의 마지막 데이터를 제거하는 동작
 - popleft : 덱의 처음 데이터를 제거하는 동작

17. 알고리즘(2)

17.1 그래프

- 그래프는 관계를 표현하는 구조
 - 그래프는 트리(Tree) 구조를 이해하는 데 필요함.
 - 노드(Node)와 엣지(Edge)로 구성된 자료 구조
- 그래프는 노드(Node)와 엣지(Edge)로 구성된 자료구조
 - 노드(Node): 그래프에서 하나의 점(데이터, 개체)
 - 엣지(Edge): 두 노드간의 연결 관계를 나타내는 데이터 (연결선)

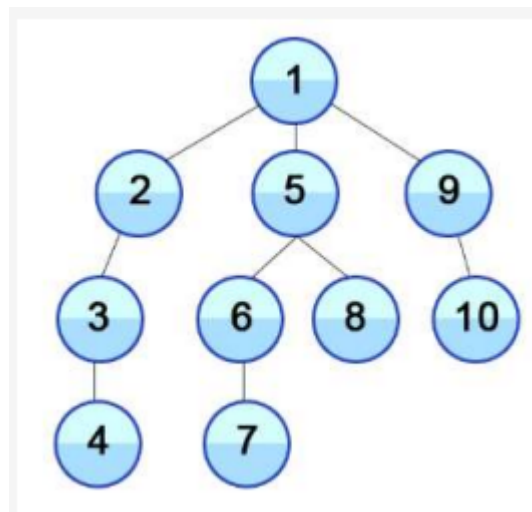
• DFS & BFS

탐색 방법	설명	자료구조	특징
DFS (깊이 우선 탐색)	한 경로를 끝까지 탐색한 후, 돌아와서 다음 경로를 탐색	스택(Stack) 또는 재귀(Recursion)	백트래킹(Backtracking) 방식, 경로 탐색에 유리
BFS (너비 우선 탐색)	가까운 노드부터 탐색한 후, 점점 멀리 탐색	큐(Queue)	최단 거리 탐색에 유리

• DFS(Depth First Search, 깊이 우선 탐색)

- 미로를 탐험할 때, 한 방향으로 쭉 가보고 막히면 되돌아와서 다른 길을 찾아가는 방식
- **특징:** 한 경로를 끝까지 탐색한 후(깊이 우선) 다른 경로를 탐색.
- 스택 이용 구현

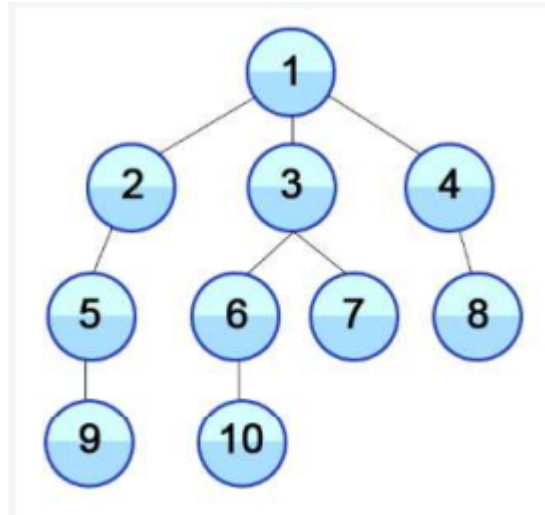
→ 데이터 순서는 아래 그림 번호 순으로 간다



• BFS(Breadth First Search, 너비 우선 탐색)

- 노드에 인접한 모든 엣지를 따라 노드에 방문하고, 다시 방문한 노드에 인접한 모든 엣지를 따라
노드에 방문하는 과정을 반복적으로 수행하며 탐색하는 방법
- 큐 이용 구현

→ 데이터 순서는 아래 그림 번호 순으로 간다



18. 그래픽스 지원 API

- 터틀 그래픽스(거북이 그래픽, Turtle Graphics)
 - 컴퓨터에 그래픽을 표시하는 하나의 방식
- 스택과 큐를 병합한 형태의 자료구조

```
import turtle
turtle.setup(410, 310)
# 배경색 변경
turtle.bgcolor('black')

# Turtle 클래스 객체 생성
star = turtle.Turtle()
star.speed(1)
star.pensize(2)

colors = ['red', 'yellow', 'blue', 'green', 'purple']

def drawStar():
    n = 5
    for i in range(n):
        star.color(colors[i])
        star.forward(100) # 앞으로 이동
```

```
star.right(144) #오른쪽 144도 회전
```

```
drawStar()  
star.hideturtle()  
# 윈도우 클릭 시 종료  
turtle.exitonclick()
```

19. 딥러닝 파이썬 패키지(1)

19.1 Pandas

- 데이터 분석과 조작을 위한 Python 라이브러리
- csv 파일 등의 데이터를 읽고 원하는 데이터 형식으로 변환
- Pandas 설치 pip 명령어로 설치 가능

```
pip install pandas
```

- Series(시리즈)
 - 1차원 배열, 라벨이 있는 데이터

```
import pandas as pd  
  
data = {'a':10, 'b':20, 'c':30}  
series = pd.Series(data)  
print(series)
```

- key를 인덱스 값으로 사용

```
• 0    10  
• 1    20  
• 2    30  
• dtype: int64
```

- DataFrame(데이터프레임)
 - 2차원 테이블 구조, 라벨이 있는 행과 열

```
import pandas as pd

data = [
    [1, 'A', 30],
    [2, 'B', 25],
    [3, 'C', 45],
]

df = pd.DataFrame(data, columns=['ID','Name','Age'])
print(d
```

- key를 컬럼 명으로 사용

	ID	Name	Age
0	1	Alice	30
1	2	Bob	35
2	3	Charlie	25

- 판다스의 주요 메세드

```
import pandas as pd

#csv 파일을 불러오는 메서드
df = pd.read_csv(파일경로)

# 데이터 상위 5개 행 보기
print(df_csv.head())

# 데이터 요약 정보
print(df.info())

# 데이터 정렬, 값으로 정렬
sorted_df = df.sort_values(by='Age')

# 열 추가
df['Salary'] = [50000, 60000, 70000]

# 행 추가
```

```

df.loc[len(df)] = [4, 'David', 40, 80000]

# 행 삭제
df = df.drop(1)

# 결측치 확인
print(df.isnull().sum())

# 결측치 채우기
meanVal = merged['Salary'].mean()
merged['Salary'] = merged['Salary'].fillna(meanVal)

# 중복 제거
df1_1 = df1.drop_duplicates()

```

19.2 Numpy(넘파이, Numerical Python)

- 다차원 배열 및 행렬 연산 지원 라이브러리
- 과학 계산을 위한 다양한 함수 제공
- numpy 설치

```
pip install numpy
```

- 판다스의 주요 메세드

```

import numpy as np

# 배열 생성
arr = np.array([1, 2, 3, 4, 5])

# 배열 속성 확인
print(arr.shape) # 배열의 모양
print(arr.dtype) # 데이터 타입

# 배열 간 연산
arr1 = np.array([1, 2, 3])
arr2 = np.array([[1], [2], [3]])
result = arr1 + arr2

```

```
# 행렬 곱:
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])
result = np.dot(matrix1, matrix2)

# 조건 필터링:
arr = np.array([1, 2, 3, 4, 5])
filtered = arr[arr > 3]
```

19.3 matplotlib

- 데이터 시각화 라이브러리
 - Matplotlib : 기본적이고 유연한 시각화 도구
 - Seaborn : 통계적 데이터 시각화에 적합

- **matplotlib** 설치

```
pip install matplotlib
```

- 시각화 예제

```
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
plt.plot(x, y) #그래프 값 넣음
plt.title("Line Plot") #해당 시각화 그래프 제목 정함
plt.xlabel("X-axis") #x축 명 정의
plt.ylabel("Y-axis") #y축 명 정의
plt.show() #그래프를 출력
```

- 다양한 그래프 종류
 - Bar Chart (막대 그래프) : 범주가 있는 데이터 값을 직사각형의 막대로 표현하는 그래프
 - Histogram (히스토그램) : 도수분포표를 그래프로 표현. 가로축은 계급, 세로축은 도수(횟수나 개수)

- Scatter Plot (산점도) : 두 변수의 상관 관계를 직교 좌표계의 평면에 점으로 표현하는 그래프
- Pie Chart (파이 차트, 원 그래프) : 범주별 구성 비율을 원형으로 표현한 그래프
- Box Plot (박스 플롯) : 수치 데이터를 표현하는 하나의 방식, 전체 데이터로부터 얻어진 다섯 가지 요약 수치를 사용

20. 딥러닝 파이썬 패키지(2)

20.1 statsmodels

- 통계적 모델 추정 및 통계적 테스트 실시 및 통계적 데이터 탐색을 위한 클래스와 함수를 제공
- R 스타일 공식과 pandas 데이터프레임을 사용한 모델 지정 지원
- 주요기능
 - 선형 및 로지스틱 회귀분석 : 데이터의 종속 변수와 한 개 이상의 독립 변수 간의 관계를 모델링
 - 시계열 분석
 - ARIMA(자기회귀 누적 이동 평균) 모델을 포함한 시계열 분석을 위한 모델 제공
 - 시계열 : 시간의 흐름에 따라 기록된 것
 - 통계적 테스트 : 가설 검정(검사 및 결정), 모델 선택, 변수 선택을 위한 테스트 제공
 - 데이터 탐색 및 시각화 : 데이터의 기초 통계량 계산 및 시각화를 통한 데이터 패턴 탐색

20.2 선형 회귀(Linear regression)

- 회귀란 레이블이 포함된 데이터를 통해 생성한 모델에 새로운 데이터를 입력하고 결과값을 예측하는 것
- 독립 변수와 종속 변수 간의 선형 관계를 모델링하는 기법
 - 독립 변수 : 독립된 변수 어떠한 것에 영향을 받지 않는 변수
 - 종속 변수 : 독립 변수에 의해 값이 변하는 수
- 선형 회귀에서는 선의 형태가 어떻게 이루어 지는지 찾는 것이 학습을 의미
- 선형회귀 방정식

$$H(x) = Wx + b$$

```
# 설치
pip install statsmodels
```

20.3 scikit-learn 패키지 (1)

- python 기반의 머신러닝 라이브러리
- 다양한 전처리, 모델링, 평가 도구 제공

```
# 설치
pip install scikit-learn
```

- 배타적 논리합(XOR)
 - 두 입력 중 하나만 참이고, 다른 한 쪽이 거짓일 때 참
 - 모두 참이거나 모두 거짓인 경우는 거짓

P	Q	P xor Q
0	0	0
1	0	1
0	1	1
1	1	0

20.4 scikit-learn 패키지 (2)

- 붓꽃의 품종 분류하기

```
from sklearn import svm, metrics

# sklearn 패키지 내 분류 알고리즘 svm 모듈로부터 SVC 클래스 사용
clf = svm.SVC()
# 주어진 훈련 데이터에 따라 모델을 피팅
clf.fit(train_data, train_label)
```

```
# 시험 데이터에 대해 분류를 수행
pre = clf.predict(test_data)
```

20.5 scipy 패키지

- Python의 과학 및 공학 계산 라이브러리
- NumPy와 함께 사용되어 데이터 분석 및 수학적 계산에 적합
- 주요 서브모듈: linalg(선형대수), optimize(최적화), stats(통계), integrate(적분), sparse(희소행렬)

```
# 설치
pip install scipy
```

- 선형 대수(linear algebra) 계산 (scipy.linalg)

```
# 선형 방정식 풀기
from scipy.linalg import solve
# Ax = b 형태의 선형 방정식
# 3x + 1y = 9
# 1x + 2y = 8
A = [[3, 1], [1, 2]]
b = [9, 8]
x = solve(A, b)
print(f"Solution: {x}")
```